



Agentic AI Bootcamp



Learn to Build AI That Thinks, Plans, and Acts.



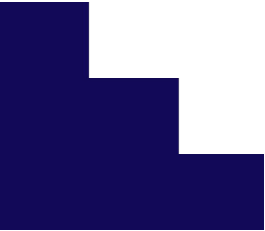
Agentic Frameworks and Orchestration

Prepared by WeCloudData





Agenda



01

Graph-based Workflows

02

Agent Orchestration

03

Framework Patterns

04

When to Use Frameworks vs Custom Code



Why Orchestration Matters

The Limit of a Single Agent

A single LLM agent no matter how well-prompted has hard limits. It has a finite context window, a single area of competence, and no ability to parallelize work.

What happens without orchestration:

- All logic lives in one massive prompt, brittle, expensive, and hard to debug
- Agent tries to be a researcher, writer, coder, and reviewer simultaneously thus quality drops across all roles
- One failure breaks the entire task ; no isolation or recovery
- No parallelism ; long tasks run sequentially even when they don't need to





Why Orchestration Matters

The Limit of a Single Agent

What orchestration adds:

- Specialized agents own specific parts of the task
- Failure in one agent is contained , others continue
- Work can be distributed and parallelized
- A coordinator maintains the big picture while workers handle details
- The system can scale horizontally; add more agents for more complex tasks

The mental model shift: Stop thinking of an AI system as *one smart agent*. Start thinking of it as *a team of agents with defined roles, communication channels, and a shared goal*.

Graph-based Workflows



Graph-Based Workflows

Why Graphs Beat Linear Pipelines

The earliest multi-step AI systems used linear pipelines: Step 1 → Step 2 → Step 3. This is simple but fails to capture real-world task complexity.

Problems with linear pipelines:

- Cannot model branching (if step 2 fails, do step 2b instead)
- Cannot model parallelism (steps 3 and 4 are independent -run them together)
- Cannot model loops with conditions (repeat step 2 until quality threshold met)
- Hard to visualize, test, and maintain as complexity grows

Why graphs solve this: A directed graph represents the workflow as nodes (agents or actions) and edges (transitions between them). Edges can be conditional, allowing the graph to branch based on runtime results.





Graph-Based Workflows

Why Graphs Beat Linear Pipelines

A graph can express:

- Sequential execution ($A \rightarrow B \rightarrow C$)
- Parallel branches ($A \rightarrow B$ and $A \rightarrow C$ simultaneously $\rightarrow D$ waits for both)
- Conditional routing ($A \rightarrow B$ if condition is true, $A \rightarrow C$ if false)
- Bounded loops ($A \rightarrow B \rightarrow A$ until criterion met, with a limit)
- Error paths ($A \rightarrow B$ on success, $A \rightarrow$ Error Handler on failure)





Graph-Based Workflows

Structure and State

Nodes Each node in an agent graph is a unit of work. It can be:

- An LLM call (with a specific system prompt and role)
- A tool call (web search, code execution, API call)
- A router (decides which edge to follow next)
- A human-in-the-loop checkpoint (waits for user input before continuing)

Edges Edges define transitions between nodes. Types:

- **Direct edge** - always go from A to B
- **Conditional edge** - go to B or C based on a condition evaluated at runtime
- **Fan-out edge** - go from A to B and C simultaneously (parallel)
- **Fan-in edge** - wait for B and C to complete before proceeding to D

State A shared state object travels through the graph. Every node reads from it and writes back to it. **State is the memory of the workflow**, it accumulates context, results, and metadata as execution proceeds.



Graph-Based Workflows

Structure and State

Nodes Each node in an agent graph is a unit of work. It can be:

- An LLM call (with a specific system prompt and role)
- A tool call (web search, code execution, API call)
- A router (decides which edge to follow next)
- A human-in-the-loop checkpoint (waits for user input before continuing)

Edges Edges define transitions between nodes. Types:

- **Direct edge** - always go from A to B
- **Conditional edge** - go to B or C based on a condition evaluated at runtime
- **Fan-out edge** - go from A to B and C simultaneously (parallel)
- **Fan-in edge** - wait for B and C to complete before proceeding to D



Graph-Based Workflows

Structure and State

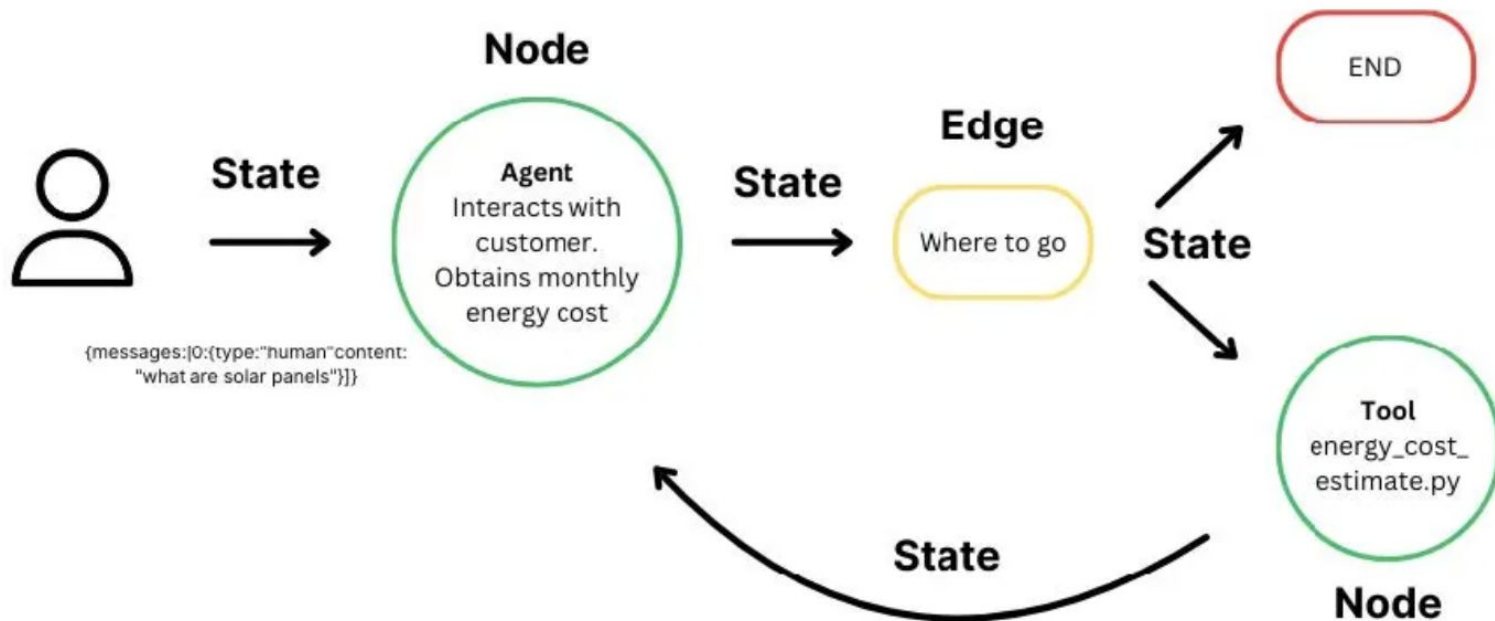
State A shared state object travels through the graph. Every node reads from it and writes back to it. **State is the memory of the workflow**, it accumulates context, results, and metadata as execution proceeds.

```
# Example state structure (LangGraph-style)
state = {
    "goal": "Research and summarize AI agent frameworks",
    "search_results": [],
    "drafted_summary": None,
    "review_feedback": None,
    "final_output": None,
    "step_count": 0
}
```



Graph-Based Workflows

Structure and State





Demo: Graph-Based Agent Workflow



Agent Orchestration



Agent Orchestration: Patterns and Roles

How Agents Are Coordinated

Orchestration is the discipline of coordinating multiple agents so they work toward a shared goal without conflicting, duplicating effort, or losing context.

The two fundamental roles in any multi-agent system:

Orchestrator (also called Supervisor, Coordinator, or Planner)

- Holds the big picture, knows the overall goal
- Decides which agent to call next
- Routes results from one agent to the next
- Handles failure and replanning
- Does not do domain work itself - it delegates



Agent Orchestration: Patterns and Roles

How Agents Are Coordinated

Worker (also called Sub-agent, Specialist, or Executor)

- Has a narrow, well-defined job
- Receives a specific task with specific inputs
- Returns a specific output
- Has no awareness of the broader goal
- Can be swapped out or upgraded without changing the orchestrator

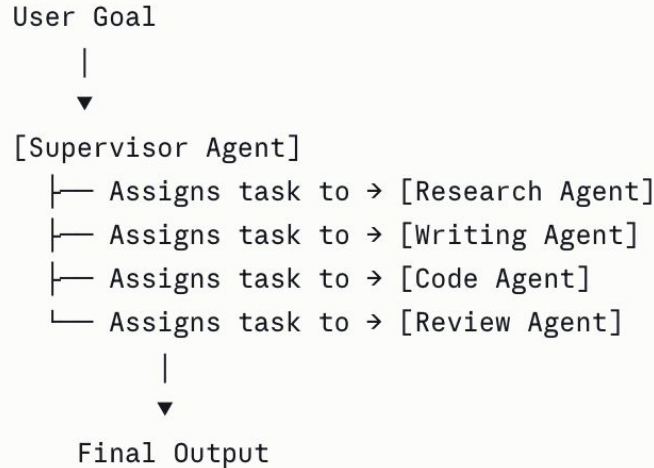
Why separation of concerns matters: When an orchestrator tries to also do domain work, it loses clarity. When a worker tries to manage the overall plan, it gets confused. Role clarity is the foundation of reliable multi-agent systems.



Agent Orchestration: Patterns and Roles

The Supervisor Pattern

One orchestrator agent directs a team of specialized worker agents. The supervisor decides what each worker does and when.





Agent Orchestration: Patterns and Roles

The Supervisor Pattern

When to use it:

- Tasks with clearly separable sub-domains (research, writing, coding, review)
- When you need centralized control and easy auditability
- When workers are independent ,one worker's output doesn't depend on real-time collaboration with another

Tradeoffs:

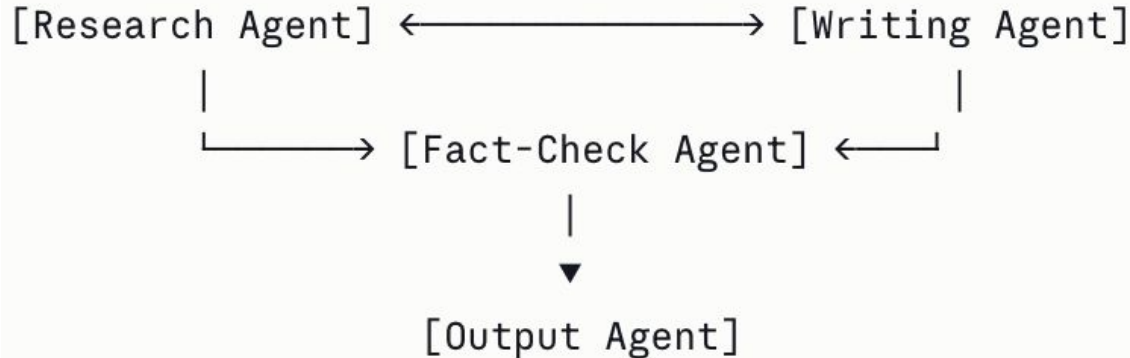
- Supervisor is a single point of failure
- Supervisor becomes a bottleneck for very large task loads
- Workers have no awareness of each other's context , can produce conflicting outputs if not managed carefully



Agent Orchestration: Patterns and Roles

The Peer-to-Peer Pattern

Agents communicate directly with each other without a central coordinator. Each agent can initiate conversations with any other agent based on its own reasoning.





Agent Orchestration: Patterns and Roles

The Peer-to-Peer Pattern

When to use it:

- Tasks requiring iterative back-and-forth between specialties (e.g., a writer and a researcher refining content together)
- Creative or exploratory tasks where the path is not known in advance
- When agents need to negotiate or debate to reach a conclusion

Tradeoffs:

- Harder to control and predict , emergent behavior can surprise you
- Loops and deadlocks are more likely without explicit guards
- Debugging is harder , no central log of decisions
- Very powerful for open-ended problems; fragile for structured ones



Agent Orchestration: Patterns and Roles

The Pipeline Pattern

The simplest orchestration pattern. Agents are arranged in a fixed sequence. Output of Agent 1 becomes input to Agent 2, and so on.

Structure:

```
Input → [Agent 1] → [Agent 2] → [Agent 3] → Output
```

Example — content pipeline:

```
Raw URL → [Scraper Agent] → [Summarizer Agent] → [Formatter Agent] → Newsletter  
entry
```



Agent Orchestration: Patterns and Roles

The Pipeline Pattern

When to use it:

- Tasks with a fixed, predictable sequence of transformations
- High-volume batch processing (process 1000 documents through the same pipeline)
- When you need maximum simplicity and reproducibility
- When each stage's output format is well-defined and stable

Tradeoffs:

- No flexibility , if stage 2 fails, the entire pipeline stalls
- Cannot handle tasks that require branching or iteration
- Easy to build, easy to maintain, easy to test
- Right choice for a surprising number of real business workflows



Agent Orchestration: Patterns and Roles

The Hierarchical Pattern

A tree structure of orchestrators and workers. A top-level orchestrator delegates to mid-level orchestrators, which delegate further to specialist workers.

```
[Chief Orchestrator]
├── [Research Orchestrator]
│   ├── [Web Search Agent]
│   └── [Document Reader Agent]
└── [Output Orchestrator]
    ├── [Writer Agent]
    └── [Review Agent]
```



Agent Orchestration: Patterns and Roles

The Hierarchical Pattern

When to use it:

- Very large, complex tasks that cannot be managed by a single orchestrator
- Enterprise workflows with distinct sub-domains (legal, financial, technical)
- Tasks requiring both breadth (many topics) and depth (expert analysis per topic)

Tradeoffs:

- Most powerful pattern , scales to arbitrarily complex tasks
- Also most expensive , many LLM calls at every level
- Coordination overhead grows with depth
- Requires careful design of inter-level communication contracts



DEMO: Multi-Agent Conversation Patterns



Framework Patterns



Framework Patterns

ReAct: Reason + Act

ReAct is the most foundational pattern in modern agentic frameworks.

The pattern: The agent alternates between two operations in a loop:

1. **Reason** - Generate a "Thought" about what to do next
2. **Act** - Take an action (call a tool, call another agent)
3. **Observe** - Receive the result of the action
4. Repeat until the goal is met or a termination condition is triggered

Thought: I need to find the current CEO of OpenAI.

Action: `web_search("OpenAI CEO 2025")`

Observation: Sam Altman is the CEO of OpenAI as of 2025.

Thought: I have the answer. No further action needed.

Final Answer: Sam Altman



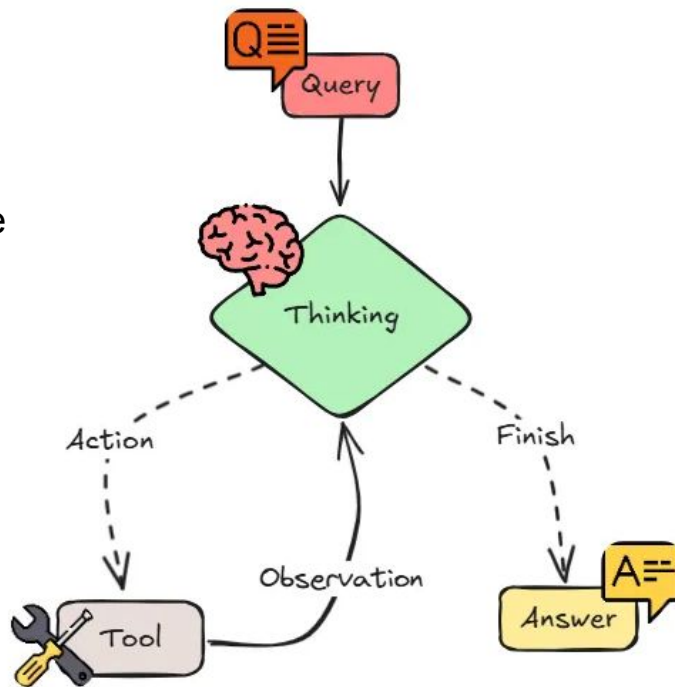
Framework Patterns

ReAct: Reason + Act

Why it works:

The explicit reasoning step forces the agent to plan the next action rather than act reflexively. This dramatically reduces errors and makes the agent's logic auditable.

ReAct Agent architecture





Framework Patterns

Plan-and-Execute: Separate Thinking from Doing

Rather than reasoning one step at a time (ReAct), Plan-and-Execute generates the full plan upfront, then executes it step by step.

The pattern:

1. **Plan phase** - One LLM call generates the complete task list
2. **Execute phase** - A separate process (or lighter LLM) executes each step
3. **Replan trigger** - If execution reveals the plan is wrong, return to step 1



Framework Patterns

Plan-and-Execute: Separate Thinking from Doing

Dimension	ReAct	Plan-and-Execute
Planning style	Incremental (one step at a time)	Upfront (full plan first)
Flexibility	High (adapts at every step)	Moderate (adapts at replan triggers)
Cost	Lower (shorter prompts per step)	Higher upfront, lower per step
Predictability	Lower	Higher
Best for	Exploratory tasks	Well-defined tasks



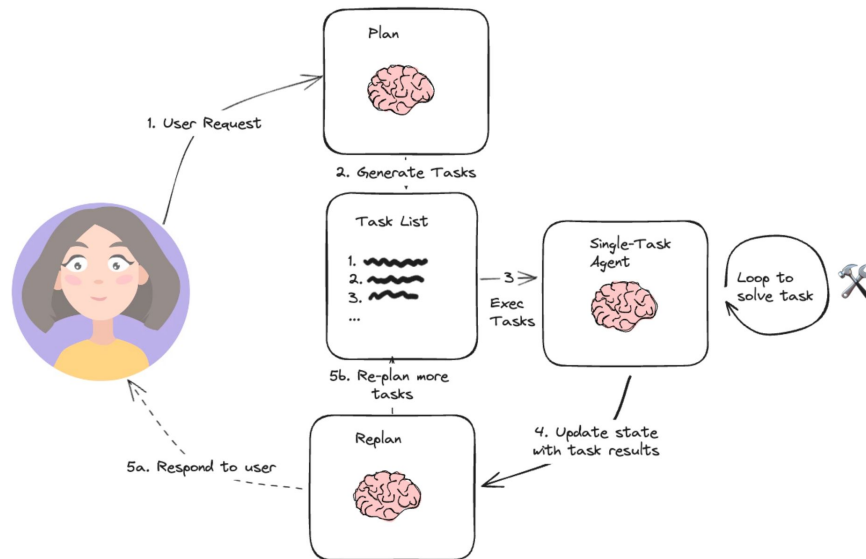


Framework Patterns

Plan-and-Execute: Separate Thinking from Doing

When to use Plan-and-Execute:

Tasks where the scope is known and stable. Long tasks where you want to review the plan before committing to execution. Tasks where execution is expensive and you want to validate the plan first.





Framework Patterns

Reflection: Agents That Review Their Own Work

Reflection is a pattern where an agent (or a separate reviewer agent) evaluates the output of a previous step and provides feedback, which is fed back into the workflow.

Two forms of reflection:

1. **Self-reflection:** The same agent that produces an output also reviews it.
2. **Cross-reflection (more robust):** A separate reviewer agent evaluates the output.

```
Agent drafts a response →  
Agent critiques its own draft →  
Agent revises based on critique →  
Output
```

```
Writer Agent drafts content →  
Reviewer Agent critiques for accuracy and clarity →  
Writer Agent revises based on feedback →  
(Loop until Reviewer approves or max iterations reached)
```




Framework Patterns

Reflection: Agents That Review Their Own Work

Reflection is a pattern where an agent (or a separate reviewer agent) evaluates the output of a previous step and provides feedback, which is fed back into the workflow.

Two forms of reflection:

1. **Self-reflection:** The same agent that produces an output also reviews it.
2. **Cross-reflection (more robust):** A separate reviewer agent evaluates the output.

```
Agent drafts a response →  
Agent critiques its own draft →  
Agent revises based on critique →  
Output
```

```
Writer Agent drafts content →  
Reviewer Agent critiques for accuracy and clarity →  
Writer Agent revises based on feedback →  
(Loop until Reviewer approves or max iterations reached)
```

When to Use Frameworks vs Custom Code



Frameworks vs Custom Code

When Should You Reach for a Framework?

This is one of the most important architectural decisions in agentic AI development. Frameworks offer speed and features. Custom code offers control and simplicity.

Use a framework when:

- You need graph-based execution, state checkpointing, or streaming out of the box
- Your team is not yet experienced with agent architecture - the framework teaches good patterns
- You need to ship quickly and the framework covers 80%+ of your requirements
- The task type has strong framework support (e.g., RAG → Haystack, multi-agent → CrewAI)
- You need observability tools like LangSmith for tracing and evaluation





Frameworks vs Custom Code

When Should You Reach for a Framework?

Use custom code when:

- The framework abstractions fight your use case more than they help
- You need performance optimizations the framework can't provide
- Your team has strong engineering skills and the problem is well-understood
- The framework introduces significant overhead (token cost, latency) for your task
- You need full control over every prompt, every call, every retry decision
- Long-term maintainability matters more than short-term speed

The pragmatic rule: Prototype with a framework. If you hit three major limitations, start extracting components to custom code. Don't rewrite from scratch - migrate incrementally.





Frameworks vs Custom Code

Frameworks vs Custom Code: Decision Matrix

Situation	Framework	Custom Code
Exploring agent design for the first time	✓	
Building a simple 2–3 step pipeline		✓
Complex branching workflow with 10+ nodes	✓	
High-volume, latency-sensitive production system		✓
Need built-in observability and tracing	✓	
Team deeply familiar with async Python		✓
Multi-agent system with role-based delegation	✓	
Simple chatbot with tool calling		✓
Regulated environment requiring full code auditability		✓
Rapid prototyping under time pressure	✓	





Frameworks vs Custom Code

Frameworks vs Custom Code: Decision Matrix

The hidden cost of frameworks

Every framework is a bet on the framework's future. LangChain's API changed significantly. Teams that built deep integrations had to refactor heavily. Factor in version lock-in risk when choosing.

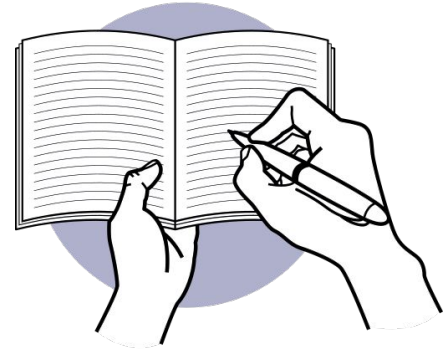
The hidden cost of custom code

You rebuild things the framework already solved - retry logic, state serialization, streaming, checkpointing. This is real engineering time.



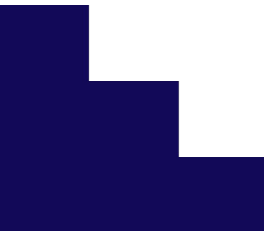
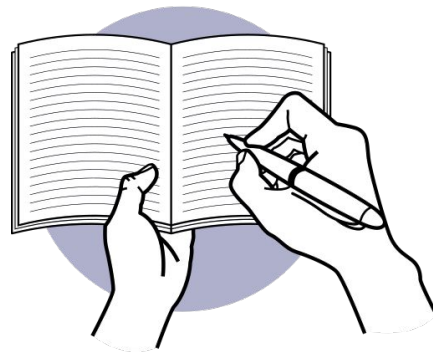


Exercise: Implement a Multi-Agent Workflow





Exercise: Add a Review Agent



Thank You



WeCloudData

500-80 Bloor Street West, Toronto

ON

www.weclouddata.com

